

Retrieval Augmented Generation (RAG)

Enterprise Guide

Overview

Retrieval Augmented Generation (RAG) is an architecture pattern that combines retrieval systems with Large Language Models (LLMs). Instead of relying only on the model's pre-trained knowledge, RAG retrieves external information and injects it into prompts before generation.

RAG improves factual accuracy, reduces hallucinations, and enables AI systems to answer questions using private or recent data.

Why RAG Is Needed

Traditional language models have limitations:

- Knowledge cutoff restrictions
- Hallucinations
- Limited context windows
- No access to private enterprise data
- Difficulty handling rapidly changing information

RAG addresses these issues by retrieving relevant information dynamically.

High Level RAG Architecture

Typical RAG workflow:

User Query

↓

Embedding Generation

↓

Vector Search

↓

Relevant Document Retrieval

↓

Context Injection



Large Language Model Response

Core Components

1. Document Ingestion

Document ingestion is the process of collecting and loading data into the system.

Common sources:

- PDF files
- Markdown documents
- Web pages
- Company knowledge bases
- SQL databases
- Support tickets
- Emails

The ingestion pipeline converts raw files into structured text.

2. Text Chunking

Large documents must be divided into smaller chunks.

Reasons:

- Context window limitations
- Better retrieval precision
- Reduced embedding costs

Chunking strategies:

Fixed Size Chunking

Example:

500 characters per chunk

Advantages:

- Simple
- Fast

Disadvantages:

- May split important context

Recursive Chunking

Recursive splitting preserves semantic boundaries.

Typical separators:

- paragraphs
- sentences
- headers

Semantic Chunking

Chunks created using meaning rather than size.

Improves retrieval quality.

3. Embeddings

Embeddings convert text into dense vectors.

Purpose:

- Represent semantic meaning
- Enable similarity search

Popular embedding models:

- all-MiniLM-L6-v2
- BAAI BGE models
- OpenAI embeddings
- E5 embeddings

Embedding dimensions vary by model.

Example:

all-MiniLM-L6-v2

Dimension Size:

384

4. Vector Databases

Vector databases store embeddings for retrieval.

Common systems:

- Qdrant
- Pinecone
- Chroma
- Weaviate
- FAISS

Capabilities:

- similarity search
 - metadata filtering
 - hybrid search
 - scalability
-

5. Retrieval

Retrieval finds relevant chunks.

Popular approaches:

Dense Retrieval

Uses embedding similarity.

Sparse Retrieval

Uses keyword matching.

Examples:

- BM25
- TF-IDF

Hybrid Retrieval

Combines:

dense retrieval + keyword retrieval

Similarity Search

Vector databases compare vectors using distance metrics.

Common metrics:

Cosine Similarity

Measures angle between vectors.

Dot Product

Measures directional similarity.

Euclidean Distance

Measures geometric distance.

Metadata Filtering

Metadata improves retrieval quality.

Examples:

department = finance

region = india

document_type = policy

Metadata allows:

filtered retrieval.

Prompt Construction

Retrieved documents are inserted into prompts.

Example:

Context:

Retrieved Chunks

Question:

User Query

Instructions:

Answer using context only.

This reduces hallucinations.

Hallucination Reduction

RAG reduces hallucinations because:

- external knowledge used
- retrieved evidence included
- prompt grounding improved

However:

RAG does NOT completely eliminate hallucinations.

Enterprise RAG Use Cases

Examples:

Customer Support

Retrieve:

- FAQs
- policies
- troubleshooting docs

Internal Knowledge Assistants

Retrieve:

- company documents
- onboarding guides
- engineering docs

Healthcare

Retrieve:

- medical policies
- research papers

Finance

Retrieve:

- compliance rules
 - reporting guidelines
-

Common Challenges

Challenges include:

Poor Chunking

Large chunks reduce precision.

Missing Metadata

Bad filtering quality.

Embedding Mismatch

Wrong model selection.

Retrieval Failures

Important chunks not found.

Latency

Large pipelines become slow.

Advanced Retrieval Techniques

Multi Query Retrieval

Generate multiple search queries.

Improves recall.

Query Expansion

Expand short queries.

Example:

"RAG"

becomes:

"Retrieval Augmented Generation architecture"

Reranking

Initial retrieval:

Top 20 chunks

Reranker:

Select best 5

Improves relevance.

Agentic RAG

Agentic RAG combines:

retrieval + agents

Agents can:

- decide retrieval strategy
- choose tools
- perform iterative search

Benefits:

- dynamic workflows
 - adaptive retrieval
-

Graph RAG

Graph RAG stores relationships.

Example:

Employee

↓

Department

↓

Projects

Benefits:

- relationship reasoning
- connected knowledge

Production RAG Pipeline

Typical enterprise pipeline:

Documents

↓

Loader

↓

Chunking

↓

Embeddings

↓

Vector DB

↓

Retrieval

↓

Prompt Assembly

↓

LLM

↓

Response

Summary

RAG combines retrieval systems with language models to create grounded AI systems. Production systems require strong ingestion pipelines, chunking strategies, embeddings, retrieval optimization, metadata filtering, and evaluation.